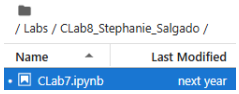
CLab 8

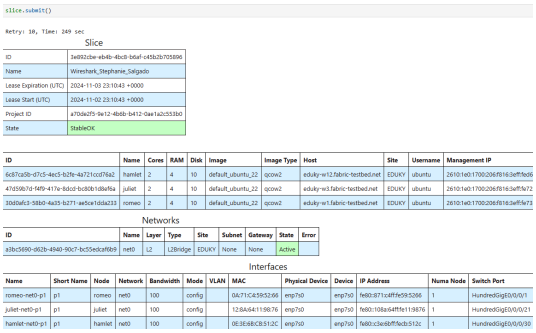
Stephanie Salgado

Setup

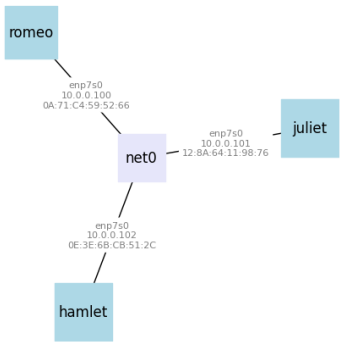


I added the same ".ipynb" file from our previous lab to a new folder for CLab8.

Submitting and viewing the slice



Since this lab follows the exact same configuration, I've skipped showing most of it. Once everything was configured, I was able to submit and view the slice.



Notice, if I draw the topology, it's exactly the same as it was for CLab7.

Getting SSH commands

Name	SSH command
romeo	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe73:5b17
juliet	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe72:25b3
hamlet	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fed6:cb8b

Like last time, I'm able to get the SSH commands by running one of the cells.

Logging into each node

```
ubuntu@romeo:~$
```

```
ubuntu@juliet:~$
```

```
ubuntu@hamlet:~$
```

As usual, I started 3 terminals and used the SHH commands to login.

Capturing network traffic with tcpdump

```
ubuntu@romeo:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:73:5b:17 brd ff:ff:ff:ff:ff:ff
    inet 10.30.9.116/19 metric 100 brd 10.30.31.255 scope global dynamic enp3s0
       valid_lft 81662sec preferred_lft 81662sec
    inet6 2610:1e0:1700:206:f816:3eff:fe73:5b17/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86330sec preferred_lft 14330sec
    inet6 fe80::f816:3eff:fe73:5b17/64 scope link
       valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 0a:71:c4:59:52:66 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.100/24 scope global enp7s0
       valid_lft forever preferred_lft forever
    inet6 fe80::871:c4ff:fe59:5266/64 scope link
       valid_lft forever preferred_lft forever
```

To begin, I use "ip addr" on "romeo" to identify the name of the interface that has the address "10.0.1.100". The interface is "enp7s0".

```
ubuntu@romeo:~$ tcpdump -i enp7s0
tcpdump: enp7s0: You don't have permission to capture on that device
(socket: Operation not permitted)

ubuntu@romeo:~$ sudo tcpdump -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Next, I use "tcpdump -i enp7s0", but I receive an error. I do it again with "sudo" and this time it begins capturing network traffic on the interface.

```
ubuntu@juliet:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.918 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.147 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.115 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.145 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.134 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4070ms
rtt min/avg/max/mdev = 0.115/0.291/0.918/0.313 ms
```

On "juliet", I begin to ping "romeo" to create some traffic.

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:39:59.892620 ARP, Request who-has romeo tell juliet, length 42
00:39:59.892659 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
00:39:59.892813 IP juliet > romeo: ICMP echo request, id 1, seq 1, length 64
00:39:59.892837 IP romeo > juliet: ICMP echo reply, id 1, seq 1, length 64
00:40:00.893181 IP juliet > romeo: ICMP echo request, id 1, seq 2, length 64
00:40:00.893205 IP romeo > juliet: ICMP echo reply, id 1, seq 2, length 64
00:40:01.914161 IP juliet > romeo: ICMP echo request, id 1, seq 3, length 64
00:40:01.914177 IP romeo > juliet: ICMP echo reply, id 1, seq 3, length 64
00:40:02.938208 IP juliet > romeo: ICMP echo request, id 1, seq 4, length 64
00:40:02.938232 IP romeo > juliet: ICMP echo reply, id 1, seq 4, length 64
00:40:03.962173 IP juliet > romeo: ICMP echo request, id 1, seq 5, length 64
00:40:03.962195 IP romeo > juliet: ICMP echo reply, id 1, seq 5, length 64
00:40:04.975626 ARP, Request who-has juliet tell romeo, length 28
00:40:04.975757 ARP, Reply juliet is-at 12:8a:64:11:98:76 (oui Unknown), length 42
```

This is what's captured. Notice there is an ARP request and reply for "romeo" when the ping starts. Then there are 5 sets (matching the 5 packets) of ICMP echo requests and replies, followed by an ARP request and reply for "juliet".

Save a packet capture to a file and review it with tcpdump and Wireshark

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0 -w romeo-tcpdump-file.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

This time, I use another command to save the capture.

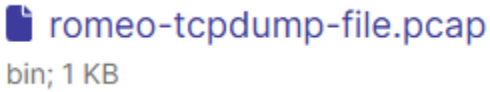
```
ubuntu@juliet:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.212 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.105 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.127 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.124 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.118 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4075ms
rtt min/avg/max/mdev = 0.105/0.137/0.212/0.038 ms
```

Once more, I ping “romeo” 5 times from “juliet”.

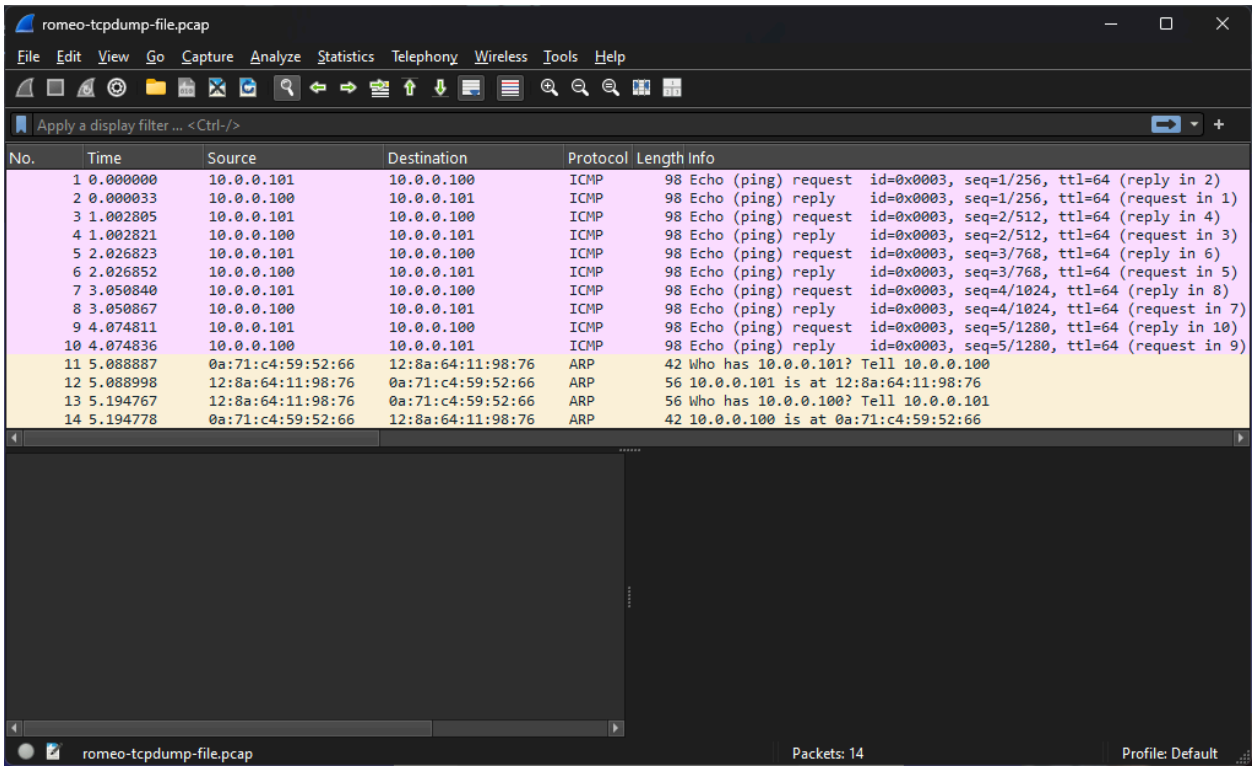
```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0 -w romeo-tcpdump-file.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C14 packets captured
14 packets received by filter
0 packets dropped by kernel
ubuntu@romeo:~$ ls
romeo-tcpdump-file.pcap
```

Notice there’s no longer a full view of the traffic captured, instead, the captured packets were saved in “rome-tcpdump-file.pcap”.

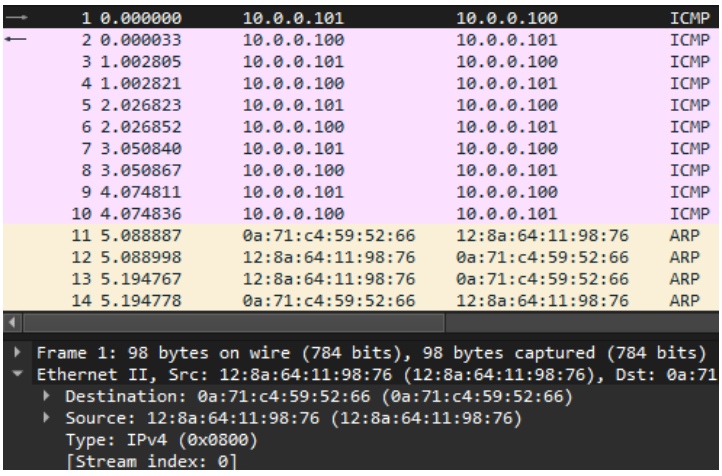
```
ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-tcpdump-file.pcap" https://file.io
{"success":true,"status":200,"id":"dbea7740-997e-11ef-b99e-f70e5612eb46","key":"8VyH8UmYCw7A","path":"/","nodeType":"file","name":"romeo-tcpdump-file.pcap"
3T00:59:20.615Z","modified":"2024-11-03T00:59:20.615Z"}ubuntu@romeo:~$
```



While the lab says to use “scp”, I’ll be using “curl -F … https://file.io” (like last time), because the transfer is seamless and extremely fast, with no timeouts.



I was able to download and view the file on Wireshark with no issues.



I click on the first packet to view details further. I notice that since the ARP requests and replies from earlier already established who “romeo” and “juliet” were, their respective MAC addresses can be seen under “Ethernet II” in “Destination” and “Source”.

Useful display options and capture options in tcpdump

```
ubuntu@romeo:~$ sudo tcpdump -enx -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

It's important to note that using "man tcpdump" can reveal all of the options tcpdump offers. In this case, using the "-enx" flags will display the link-level header (e), prevent hostname resolution (n), and print the packet payload in both hexadecimal and ASCII format (x).

```
ubuntu@romeo:~$ sudo tcpdump -enx -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:19:18.919523 12:8a:64:11:98:76 > 0a:71:c4:59:52:66, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo request, id 4, seq 1, length 64
        0x0000:  4500 0054 4d11 4000 4001 d8cf 0a00 0065
        0x0010:  0a00 0064 0800 76d8 0004 0001 96cf 2667
        0x0020:  0000 0000 f718 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:18.919552 0a:71:c4:59:52:66 > 12:8a:64:11:98:76, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo reply, id 4, seq 1, length 64
        0x0000:  4500 0054 a2cc 0000 4001 c314 0a00 0064
        0x0010:  0a00 0065 0000 7ed8 0004 0001 96cf 2667
        0x0020:  0000 0000 f718 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:19.931113 12:8a:64:11:98:76 > 0a:71:c4:59:52:66, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo request, id 4, seq 2, length 64
        0x0000:  4500 0054 4d6b 4000 4001 d875 0a00 0065
        0x0010:  0a00 0064 0800 1faa 0004 0002 97cf 2667
        0x0020:  0000 0000 4d46 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:19.931133 0a:71:c4:59:52:66 > 12:8a:64:11:98:76, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo reply, id 4, seq 2, length 64
        0x0000:  4500 0054 a31e 0000 4001 c2c2 0a00 0064
        0x0010:  0a00 0065 0000 27aa 0004 0002 97cf 2667
        0x0020:  0000 0000 4d46 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
```

I ping "romeo" from "juliet" a third time. The tcpdump output reflects this.

```
ubuntu@romeo:~$ sudo tcpdump -s 34 -w romeo-tcpdump-snaplen.pcap -i enp7s0
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 34 bytes
```

```
ubuntu@romeo:~$ ls -l
total 8
-rw-r--r-- 1 tcpdump tcpdump 1424 Nov  3 00:51 romeo-tcpdump-file.pcap
-rw-r--r-- 1 tcpdump tcpdump  724 Nov  3 01:25 romeo-tcpdump-snaplen.pcap
```

```
ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-tcpdump-snaplen.pcap" https://file.io
{"success":true,"status":200,"id":"d60ca360-9984-11ef-a8e1-8749aa288cfc","key":"hU8xrmZeRUqp","path":"/","nodeType":"file","name":"romeo-tcpdump-snaplen.pcap"
-03T01:42:07.757Z","modified":"2024-11-03T01:42:07.757Z"}ubuntu@romeo:~$
```

The "-s 34" flag will ensure that only the first 34 bytes of each packet will be captured. Even though 5 packets were transmitted again, the file is smaller.

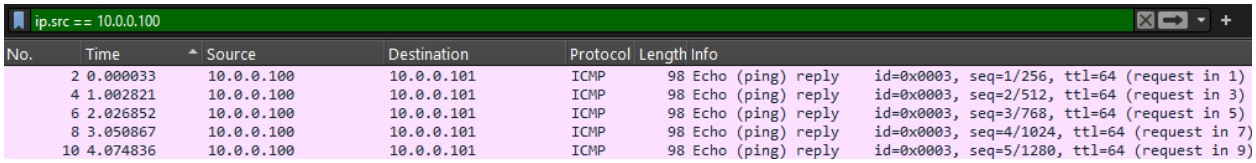
```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0 src host 10.0.0.100
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:30:17.243951 IP romeo > juliet: ICMP echo reply, id 6, seq 1, length 64
01:30:18.265431 IP romeo > juliet: ICMP echo reply, id 6, seq 2, length 64
01:30:19.289444 IP romeo > juliet: ICMP echo reply, id 6, seq 3, length 64
01:30:20.313441 IP romeo > juliet: ICMP echo reply, id 6, seq 4, length 64
01:30:21.337445 IP romeo > juliet: ICMP echo reply, id 6, seq 5, length 64
01:30:22.297402 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
01:30:22.447623 ARP, Request who-has juliet tell romeo, length 28
^C
7 packets captured
7 packets received by filter
0 packets dropped by kernel
```

```
ubuntu@romeo:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.025 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.023 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.022 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.023 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4094ms
rtt min/avg/max/mdev = 0.022/0.036/0.087/0.025 ms
```

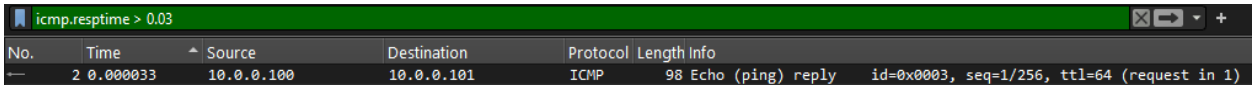
```
ubuntu@juliet:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.127 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.122 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.130 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.119 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.121 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4093ms
rtt min/avg/max/mdev = 0.119/0.123/0.130/0.004 ms
```

Since tcpdump also supports capture filters, using the filter “src host 10.0.0.100” should capture only the packets that have that source IP. I pinged “10.0.0.100” (romeo) from both “romeo” and “juliet”, yet the only packets shown were from ICMP echo reply packets from “romeo” to “juliet”.

Useful display options in Wireshark



In the first capture file (romeo-tcpdump-file.pcap), using the “ip.src == 10.0.0.100” will also only show packets that came from the source IP “10.0.0.100”. Once again, all these packets are the ICMP echo replies sent to “juliet”.

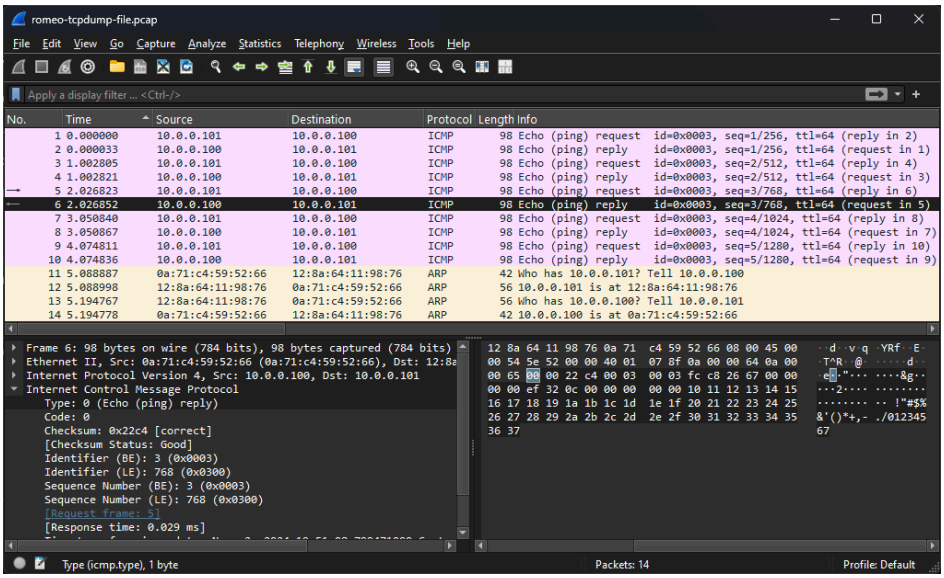


Similarly, using “icmp.resptime > 0.03” will only display the packets that had a response time greater than 0.03 seconds.

Exercises

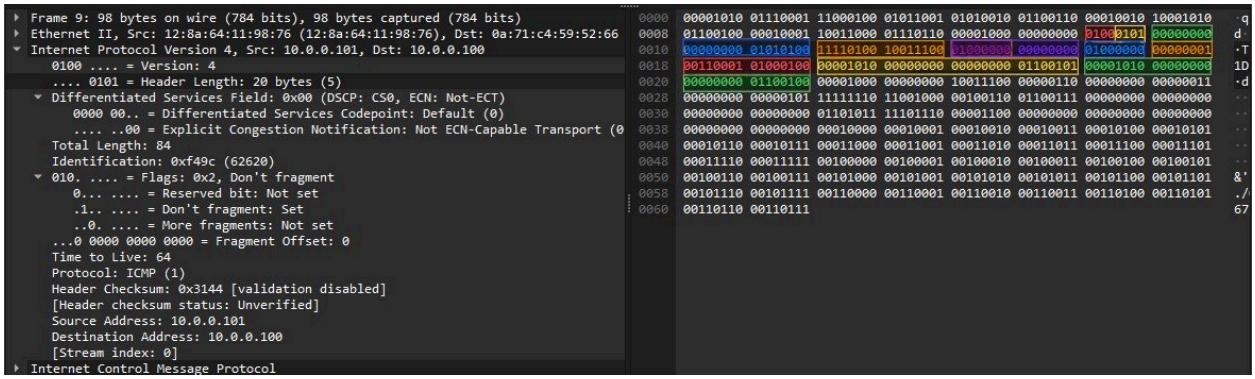
From "Save a packet capture to a file and review it with tcpdump and Wireshark":

1.



I selected an ICMP packet (6) in Wireshark and highlighted the "Type" field in the packet detail pane. The corresponding byte was also highlighted in the packet bytes pane. This field takes up 1 byte in the ICMP packet. I can tell it's 1 byte because only two hex digits are highlighted in the packet bytes pane.

2.



Using packet 9, from left to right - Version: Red (4 bits), Header Length: Yellow (4 bits), Differentiated Services Field: Green (8 bits), Total Length: Blue (16 bits), Identification: Orange (16 bits), Flags: Pink (3 bits), Fragment Offset: Purple (13 bits), Time to Live: Blue (8 bits), Protocol: Orange (8 bits), Header Checksum: Red (16 bits), Source Address: Yellow (32 bits), Destination Address: Green (32 bits).

Yes, the bits in the packet bytes pane match the values in the packet details pane. Wireshark decodes the raw binary data directly, so each IP header field has the same value in both panes.

From "Useful display options and capture options in tcpdump":

1.

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:39:59.892620 ARP, Request who-has romeo tell juliet, length 42
00:39:59.892659 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
00:39:59.892813 IP juliet > romeo: ICMP echo request, id 1, seq 1, length 64
00:39:59.892837 IP romeo > juliet: ICMP echo reply, id 1, seq 1, length 64
00:40:00.893181 IP juliet > romeo: ICMP echo request, id 1, seq 2, length 64
00:40:00.893205 IP romeo > juliet: ICMP echo reply, id 1, seq 2, length 64
00:40:01.914161 IP juliet > romeo: ICMP echo request, id 1, seq 3, length 64
00:40:01.914177 IP romeo > juliet: ICMP echo reply, id 1, seq 3, length 64
00:40:02.938208 IP juliet > romeo: ICMP echo request, id 1, seq 4, length 64
00:40:02.938232 IP romeo > juliet: ICMP echo reply, id 1, seq 4, length 64
00:40:03.962173 IP juliet > romeo: ICMP echo request, id 1, seq 5, length 64
00:40:03.962195 IP romeo > juliet: ICMP echo reply, id 1, seq 5, length 64
00:40:04.975626 ARP, Request who-has juliet tell romeo, length 28
00:40:04.975757 ARP, Reply juliet is-at 12:8a:64:11:98:76 (oui Unknown), length 42
```

```
ubuntu@romeo:~$ sudo tcpdump -enx -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:19:18.919523 12:8a:64:11:98:76 > 0a:71:c4:59:52:66, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo request, id 4, seq 1, length 64
        0x0000:  4500 0054 4d11 4000 4001 d8cf 0a00 0065
        0x0010:  0a00 0064 0800 76d8 0004 0001 96cf 2667
        0x0020:  0000 0000 f718 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:18.919552 0a:71:c4:59:52:66 > 12:8a:64:11:98:76, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo reply, id 4, seq 1, length 64
        0x0000:  4500 0054 a2cc 0000 4001 c314 0a00 0064
        0x0010:  0a00 0065 0000 7ed8 0004 0001 96cf 2667
        0x0020:  0000 0000 f718 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:19.931113 12:8a:64:11:98:76 > 0a:71:c4:59:52:66, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo request, id 4, seq 2, length 64
        0x0000:  4500 0054 4d6b 4000 4001 d875 0a00 0065
        0x0010:  0a00 0064 0800 1faa 0004 0002 97cf 2667
        0x0020:  0000 0000 4d46 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
01:19:19.931133 0a:71:c4:59:52:66 > 12:8a:64:11:98:76, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo reply, id 4, seq 2, length 64
        0x0000:  4500 0054 a31e 0000 4001 c2c2 0a00 0064
        0x0010:  0a00 0065 0000 27aa 0004 0002 97cf 2667
        0x0020:  0000 0000 4d46 0e00 0000 0000 1011 1213
        0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223
        0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233
        0x0050:  3435 3637
```

In the first screenshot, using “sudo tcpdump -i enp7s0”, the output shows ICMP echo requests and replies between "juliet" and "romeo," with details like the protocol, source and destination hostnames, and ICMP packet information such as id, seq, and packet length. A few ARP messages are also included, with MAC addresses partially listed as Unknown. In the second screenshot, I used “sudo tcpdump -enx -i enp7s0”, which added more details (I discussed this above).

2.

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:39:59.892620 ARP, Request who-has romeo tell juliet, length 42
00:39:59.892659 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
00:39:59.892813 IP juliet > romeo: ICMP echo request, id 1, seq 1, length 64
00:39:59.892837 IP romeo > juliet: ICMP echo reply, id 1, seq 1, length 64
00:40:00.893181 IP juliet > romeo: ICMP echo request, id 1, seq 2, length 64
00:40:00.893205 IP romeo > juliet: ICMP echo reply, id 1, seq 2, length 64
00:40:01.914161 IP juliet > romeo: ICMP echo request, id 1, seq 3, length 64
00:40:01.914177 IP romeo > juliet: ICMP echo reply, id 1, seq 3, length 64
00:40:02.938208 IP juliet > romeo: ICMP echo request, id 1, seq 4, length 64
00:40:02.938232 IP romeo > juliet: ICMP echo reply, id 1, seq 4, length 64
00:40:03.962173 IP juliet > romeo: ICMP echo request, id 1, seq 5, length 64
00:40:03.962195 IP romeo > juliet: ICMP echo reply, id 1, seq 5, length 64
00:40:04.975626 ARP, Request who-has juliet tell romeo, length 28
00:40:04.975757 ARP, Reply juliet is-at 12:8a:64:11:98:76 (oui Unknown), length 42
```

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0 src host 10.0.0.100
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
01:30:17.243951 IP romeo > juliet: ICMP echo reply, id 6, seq 1, length 64
01:30:18.265431 IP romeo > juliet: ICMP echo reply, id 6, seq 2, length 64
01:30:19.289444 IP romeo > juliet: ICMP echo reply, id 6, seq 3, length 64
01:30:20.313441 IP romeo > juliet: ICMP echo reply, id 6, seq 4, length 64
01:30:21.337445 IP romeo > juliet: ICMP echo reply, id 6, seq 5, length 64
01:30:22.297402 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
01:30:22.447623 ARP, Request who-has juliet tell romeo, length 28
^C
7 packets captured
7 packets received by filter
0 packets dropped by kernel
```

In these screenshots, I used tcpdump with and without a capture filter for “src host 10.0.0.100”. In the first screenshot, using “sudo tcpdump -i enp7s0”, I captured all traffic, including both ICMP packets and ARP messages between "romeo" and "juliet." In the second screenshot, I used “sudo tcpdump -i enp7s0 src host 10.0.0.100”, filtering for packets from 10.0.0.100 only. This output shows only ICMP replies from 10.0.0.100 and no ARP messages, focusing on relevant traffic and reducing unnecessary data (I also mentioned this above).

3.

```
ubuntu@romeo:~$ sudo tcpdump -i enp7s0 host juliet and not icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
03:33:06.399781 ARP, Request who-has romeo tell juliet, length 42
03:33:06.399797 ARP, Reply romeo is-at 0a:71:c4:59:52:66 (oui Unknown), length 28
03:33:06.543634 ARP, Request who-has juliet tell romeo, length 28
03:33:06.543699 ARP, Reply juliet is-at 12:8a:64:11:98:76 (oui Unknown), length 42
^C
4 packets captured
4 packets received by filter
0 packets dropped by kernel
```

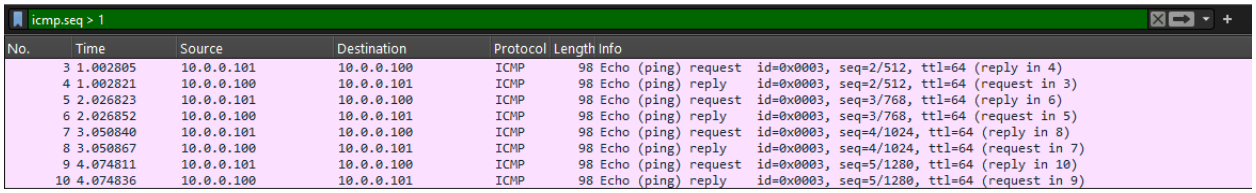
```
ubuntu@romeo:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.023 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.029 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.027 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.028 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.029 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4103ms
rtt min/avg/max/mdev = 0.023/0.027/0.029/0.002 ms
ubuntu@romeo:~$ ping -c 5 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
64 bytes from 10.0.0.101: icmp_seq=1 ttl=64 time=0.104 ms
64 bytes from 10.0.0.101: icmp_seq=2 ttl=64 time=0.105 ms
64 bytes from 10.0.0.101: icmp_seq=3 ttl=64 time=0.104 ms
64 bytes from 10.0.0.101: icmp_seq=4 ttl=64 time=0.098 ms
64 bytes from 10.0.0.101: icmp_seq=5 ttl=64 time=0.118 ms

--- 10.0.0.101 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4072ms
rtt min/avg/max/mdev = 0.098/0.105/0.118/0.006 ms
```

```
ubuntu@juliet:~$ ping -c 5 10.0.0.100
PING 10.0.0.100 (10.0.0.100) 56(84) bytes of data.
64 bytes from 10.0.0.100: icmp_seq=1 ttl=64 time=0.146 ms
64 bytes from 10.0.0.100: icmp_seq=2 ttl=64 time=0.111 ms
64 bytes from 10.0.0.100: icmp_seq=3 ttl=64 time=0.108 ms
64 bytes from 10.0.0.100: icmp_seq=4 ttl=64 time=0.109 ms
64 bytes from 10.0.0.100: icmp_seq=5 ttl=64 time=0.104 ms

--- 10.0.0.100 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4102ms
rtt min/avg/max/mdev = 0.104/0.115/0.146/0.015 ms
ubuntu@juliet:~$ ping -c 5 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
64 bytes from 10.0.0.101: icmp_seq=1 ttl=64 time=0.016 ms
64 bytes from 10.0.0.101: icmp_seq=2 ttl=64 time=0.023 ms
64 bytes from 10.0.0.101: icmp_seq=3 ttl=64 time=0.030 ms
64 bytes from 10.0.0.101: icmp_seq=4 ttl=64 time=0.023 ms
64 bytes from 10.0.0.101: icmp_seq=5 ttl=64 time=0.035 ms

--- 10.0.0.101 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4089ms
rtt min/avg/max/mdev = 0.016/0.025/0.035/0.006 ms
```

The command would be “sudo tcpdump -i enp7s0 host juliet and not icmp”. I used the command and then pinged between “romeo” and “juliet”. Notice only ARP packets were captured, this is because of “not icmp” which filters out all the ICMP packets generated by the pings.

From "Useful display options in Wireshark":

1.



icmp.seq > 1

No.	Time	Source	Destination	Protocol	Length	Info
3	1.002805	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=64 (reply in 4)
4	1.002821	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) reply id=0x0003, seq=2/512, ttl=64 (request in 3)
5	2.026823	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=64 (reply in 6)
6	2.026852	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) reply id=0x0003, seq=3/768, ttl=64 (request in 5)
7	3.050840	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=64 (reply in 8)
8	3.050867	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) reply id=0x0003, seq=4/1024, ttl=64 (request in 7)
9	4.074811	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) request id=0x0003, seq=5/1280, ttl=64 (reply in 10)
10	4.074836	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) reply id=0x0003, seq=5/1280, ttl=64 (request in 9)

To filter packets in Wireshark to display only those with an ICMP sequence number greater than 1, the filter would be “icmp.seq > 1”.

Deleting the slice

Slice

ID	3e892cbe-eb4b-4bc8-b6af-c45b2b705896
Name	Wireshark_Stephanie_Salgado
Lease Expiration (UTC)	2024-11-03 23:10:43 +0000
Lease Start (UTC)	2024-11-02 23:10:43 +0000
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
State	Dead

Finally, I can delete the slice.

Summary

For this lab, I used the same network topology from the previous lab. This time, I was able to inspect network traffic through tcpdump and Wireshark. I used different flags and capture filters to perfectly decide what information I wanted to capture and display. I was also able to transfer the files from “romeo” to my own desktop using “file.io” like in CLab7. Furthermore, I was able to view those files in Wireshark and apply different filters to achieve the same effect of flags and filters on tcpdump. I was also able to complete the “Exercises” section with no issues. Overall, this lab was good to get a good idea of how to use tcpdump and Wireshark filters.